Problemas NP-completos

Programación III - UNGS

Problemas NP-completos

- Sean A y B dos problemas de decisión (es decir, problemas cuya solución es **true** o **false**).

Problemas NP-completos

- Sean A y B dos problemas de decisión (es decir, problemas cuya solución es **true** o **false**).
- **Definición:** Una transformación de A en B es un programa p que toma un input I para A y lo convierte en un input $p(I) = I'$ para B, de modo tal que $A(I) =$ **true** si y sólo si $B(I') =$ **true**.

Problemas NP-completos

- Sean A y B dos problemas de decisión (es decir, problemas cuya solución es **true** o **false**).
- **Definición:** Una transformación de A en B es un programa p que toma un input I para A y lo convierte en un input $p(I) = I'$ para B, de modo tal que $A(I) =$ **true** si y sólo si $B(I') =$ **true**.
- Si p es un algoritmo polinomial en el tamaño de I, decimos que es una transformación polinomial.

Problemas NP-completos

- Sean A y B dos problemas de decisión (es decir, problemas cuya solución es **true** o **false**).
- **Definición:** Una transformación de A en B es un programa p que toma un input I para A y lo convierte en un input $p(I) = I'$ para B, de modo tal que $A(I) =$ **true** si y sólo si $B(I') =$ **true**.
- Si p es un algoritmo polinomial en el tamaño de I, decimos que es una transformación polinomial.
- Si existe una transformación polinomial de A en B, escribimos $A \leq_P B$.

Problemas NP-completos

- La notación $A \leq_P B$ denota que B es igual o más difícil que A, dado que se puede usar un algoritmo b que resuelve B para resolver A.

Problemas NP-completos

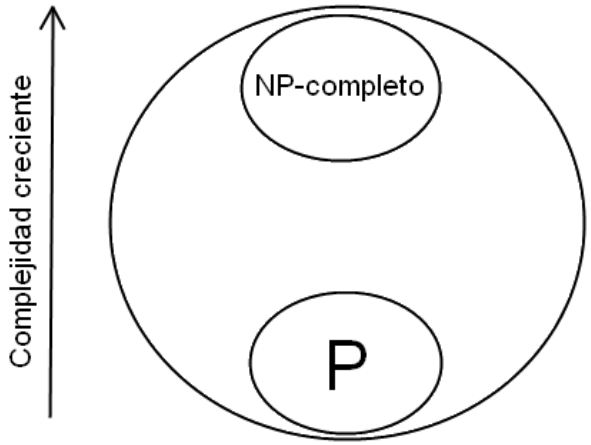
- La notación $A \leq_P B$ denota que B es igual o más difícil que A, dado que se puede usar un algoritmo b que resuelve B para resolver A.
- **Definición:** Un problema $B \in NP$ es NP-completo si $A \leq_P B$ para todo otro problema $A \in NP$.

Problemas NP-completos

- La notación $A \leq_P B$ denota que B es igual o más difícil que A, dado que se puede usar un algoritmo b que resuelve B para resolver A.
- **Definición:** Un problema $B \in NP$ es NP-completo si $A \leq_P B$ para todo otro problema $A \in NP$.
- No se sabe si existen algoritmos polinomiales para los problemas NP-completos, y la existencia de un algoritmo polinomial para uno de ellos implicaría la existencia de algoritmos polinomiales para todos ellos.

Problemas NP-completos

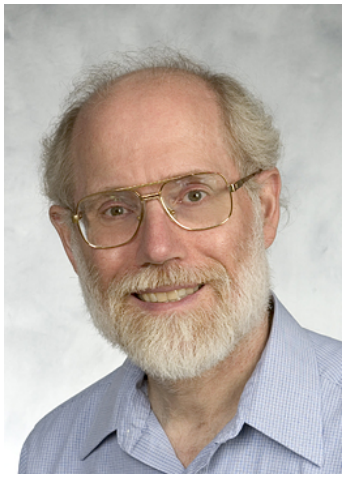
- Si llamamos P a la clase de problemas que admiten un algoritmo polinomial, la clase NP tiene la siguiente forma:



Problemas NP-completos



Michael Garey



David Johnson

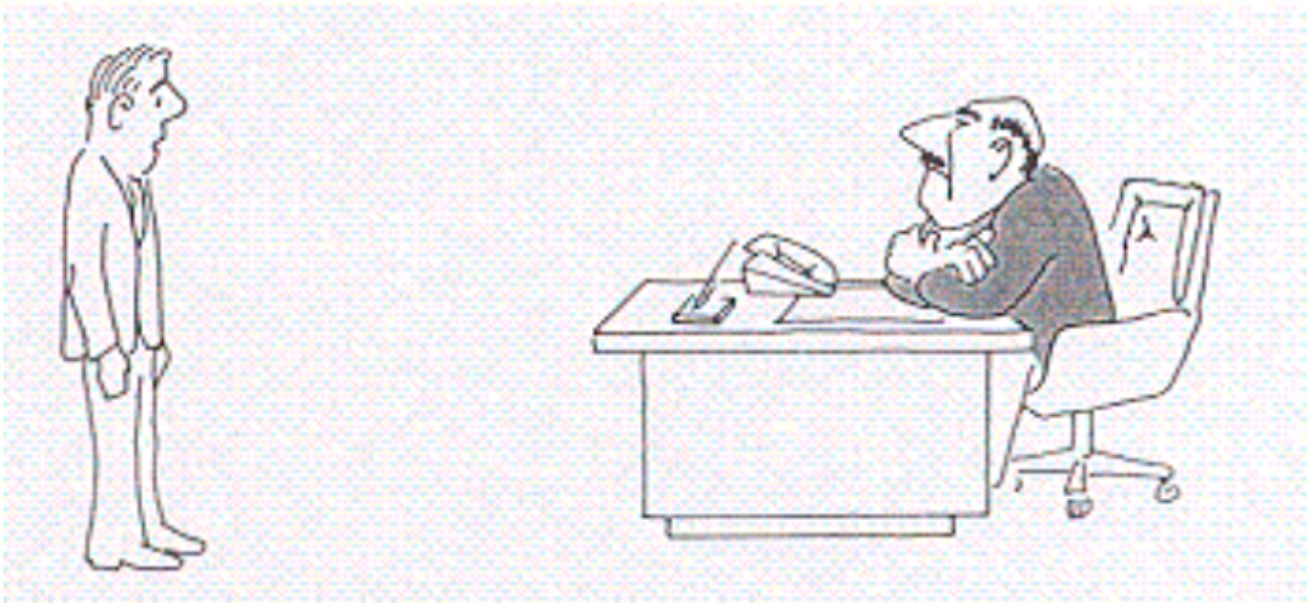
- M. Garey y D. Johnson, Computers and Intractability: A Guide to the Theory of NP-Completeness. W. H. Freeman and Company, 1979.

Problemas NP-completos

One day your boss calles you into his office and confides that the company is about to enter the highly competitive "bandersnatch" market. A good method is needed for determining whether or not any given set of specifications for a new bandersnatch component can be met and, if so, for constructing a design that meets them. Since you are the company's chief algorithm designer, your charge is to find an efficient algorithm for doing this.

After consulting with the bandersnatch department to determine exactly what the problem is, you eagerly hurry back to your office, and plunge into the task. Some weeks later, you have not been able to come up with any algorithm substantially better than searching through all possible designs. This would involve years of computation time for just one set of specifications. You certainly don't want to return to your boss's office and report:

Problemas NP-completos



"I can't find an efficient algorithm,
I guess I'm just too dumb."

Problemas NP-completos

To avoid serious damage to your position within the company, it would be much better if you could prove that the bandersnatch problem is inherently intractable, that no algorithm could possibly solve it quickly. Then you could stride confidently into the boss's office and proclaim:

Problemas NP-completos



"I can't find an efficient algorithm,
because no such algorithm is possible! "

Problemas NP-completos

Unfortunately, proving inherent intractibility can be just as hard as finding efficient algorithms.

The theory of NP-completeness provides many straightforward techniques for proving that a given problem is "just as hard" as a large number of other problems that are widely recognize as being difficult and that have been confounding the experts for years. Armed with these techniques, you might be able to prove that the bandersnatch problem is NP-complete and march into your boss's office and announce:

Problemas NP-completos



"I can't find an efficient algorithm,
but neither can all these famous people."

Problemas NP-completos

At the very least, this would inform your boss that it would do no good to fire you and hire another expert on algorithms.

Now you can spend you time looking for efficient algorithms that solve various special cases of the general problem. You might look for algorithms that, though not guaranteed to run quickly, seem likely to do so most of the time. Or you might even relax the problem somewhat, looking for a fast algorithm that merely finds designs that meet most of the component specifications. Thus, the primary application of the theory of NP-completeness is to assist algorithm designers in directing their problem-solving efforts toward those approaches that have the greatest likelihood of leading to useful algorithms.

Problemas NP-completos

- ¿Cómo se demuestra que un problema B es NP-completo?

Problemas NP-completos

- ¿Cómo se demuestra que un problema B es NP-completo?

Problemas NP-completos

- ¿Cómo se demuestra que un problema B es NP-completo? Recordemos la definición ...
- **Definición:** Un problema $B \in NP$ es NP-completo si $A \leq_P B$ para todo otro problema $A \in NP$.

Problemas NP-completos

- ¿Cómo se demuestra que un problema B es NP-completo? Recordemos la definición ...
- **Definición:** Un problema $B \in NP$ es NP-completo si $A \leq_P B$ para todo otro problema $A \in NP$.
- Una posibilidad es dar una transformación polinomial desde otro problema NP-completo C hacia nuestro problema B. Entonces, para todo problema $A \in NP$, tenemos que:

$$A \leq_P C \leq_p B.$$

Problemas NP-completos

- **Ejemplo:** Consideremos el siguiente problema.

Problemas NP-completos

- **Ejemplo:** Consideremos el siguiente problema.

  Ruteo de vehículos (VRP)

Problemas NP-completos

- **Ejemplo:** Consideremos el siguiente problema.

  Ruteo de vehículos (VRP)

  1. **Entrada:** Un conjunto de n clientes con su matriz de distancias $A \in \mathbb{R}^{n \times n}$. Una cantidad c_i a entregar en cada cliente $i = 1, \ldots, n$. Una cantidad m de camiones y la capacidad C de cada camión. Un número real $k \in \mathbb{R}$.

Problemas NP-completos

- **Ejemplo:** Consideremos el siguiente problema.

  Ruteo de vehículos (VRP)

  1. **Entrada:** Un conjunto de n clientes con su matriz de distancias $A \in \mathbb{R}^{n \times n}$. Una cantidad c_i a entregar en cada cliente $i = 1, \ldots, n$. Una cantidad m de camiones y la capacidad C de cada camión. Un número real $k \in \mathbb{R}$.
  2. **Salida:** ¿Existe un conjunto de m rutas que pase por todos los clientes, tal que la suma de las capacidades de los clientes en cada ruta no excede la capacidad C del camión, y tal que la distancia total recorrida es menor o igual que k?

Problemas NP-completos

- **Ejemplo:** Consideremos el siguiente problema.

  Ruteo de vehículos (VRP)

  1. **Entrada:** Un conjunto de n clientes con su matriz de distancias $A \in \mathbb{R}^{n \times n}$. Una cantidad c_i a entregar en cada cliente $i = 1, \ldots, n$. Una cantidad m de camiones y la capacidad C de cada camión. Un número real $k \in \mathbb{R}$.
  2. **Salida:** ¿Existe un conjunto de m rutas que pase por todos los clientes, tal que la suma de las capacidades de los clientes en cada ruta no excede la capacidad C del camión, y tal que la distancia total recorrida es menor o igual que k?

- Queremos probar que este problema es NP-completo. En primer lugar, es fácil verificar que VRP $\in$ NP (¿por qué?).

Problemas NP-completos

- El VRP tiene varios elementos en común con el TSP:

Problemas NP-completos

- El VRP tiene varios elementos en común con el TSP:

  Viajante de comercio (TSP)

Problemas NP-completos

- El VRP tiene varios elementos en común con el TSP:

  Viajante de comercio (TSP)

  1. **Entrada:** Un conjunto de n ciudades con su matriz de distancias $A \in \mathbb{R}^{n \times n}$, y un número real $k \in \mathbb{R}$.

Problemas NP-completos

- El VRP tiene varios elementos en común con el TSP:

  Viajante de comercio (TSP)

  1. **Entrada:** Un conjunto de n ciudades con su matriz de distancias $A \in \mathbb{R}^{n \times n}$, y un número real $k \in \mathbb{R}$.
  2. **Salida:** ¿Existe un recorrido que pase por todas las ciudades y que tenga distancia menor o igual que k?

Problemas NP-completos

- El VRP tiene varios elementos en común con el TSP:

  Viajante de comercio (TSP)
  1. **Entrada:** Un conjunto de n ciudades con su matriz de distancias $A \in \mathbb{R}^{n \times n}$, y un número real $k \in \mathbb{R}$.
  2. **Salida:** ¿Existe un recorrido que pase por todas las ciudades y que tenga distancia menor o igual que k?

- Veamos entonces que TSP $\leq_P$ VRP. Para esto, hay que construir una transformación polinomial de una instancia (arbitraria) del TSP en una instancia del VRP, que conserve la respuesta.

Problemas NP-completos

- Dada la siguiente instancia *I* del TSP:

Problemas NP-completos

- Dada la siguiente instancia I del TSP:
  1. Matriz de distancias $A \in \mathbb{R}^{n \times n}$.

Problemas NP-completos

- Dada la siguiente instancia I del TSP:
  1. Matriz de distancias $A \in \mathbb{R}^{n \times n}$.
  2. Número real $k \in \mathbb{R}$.

Problemas NP-completos

- Dada la siguiente instancia I del TSP:
  1. Matriz de distancias $A \in \mathbb{R}^{n \times n}$.
  2. Número real $k \in \mathbb{R}$.
- ... construimos la siguiente instancia I' del VRP:

Problemas NP-completos

- Dada la siguiente instancia I del TSP:
  1. Matriz de distancias $A \in \mathbb{R}^{n \times n}$.
  2. Número real $k \in \mathbb{R}$.
- ... construimos la siguiente instancia I' del VRP:
  1. Matriz de distancias $A \in \mathbb{R}^{n \times n}$.

Problemas NP-completos

- Dada la siguiente instancia I del TSP:
  1. Matriz de distancias $A \in \mathbb{R}^{n \times n}$.
  2. Número real $k \in \mathbb{R}$.
- ... construimos la siguiente instancia I' del VRP:
  1. Matriz de distancias $A \in \mathbb{R}^{n \times n}$.
  2. Cantidad $m = 1$ de camiones.

Problemas NP-completos

- Dada la siguiente instancia I del TSP:
  1. Matriz de distancias $A \in \mathbb{R}^{n \times n}$.
  2. Número real $k \in \mathbb{R}$.
- ... construimos la siguiente instancia I' del VRP:
  1. Matriz de distancias $A \in \mathbb{R}^{n \times n}$.
  2. Cantidad $m = 1$ de camiones.
  3. Cada cliente $i = 1, \ldots, n$ tiene capacidad $c_i = 1$.

Problemas NP-completos

- Dada la siguiente instancia I del TSP:
  1. Matriz de distancias $A \in \mathbb{R}^{n \times n}$.
  2. Número real $k \in \mathbb{R}$.
- ... construimos la siguiente instancia I' del VRP:
  1. Matriz de distancias $A \in \mathbb{R}^{n \times n}$.
  2. Cantidad $m = 1$ de camiones.
  3. Cada cliente $i = 1, \ldots, n$ tiene capacidad $c_i = 1$.
  4. El único camión tiene capacidad $C = n$.

Problemas NP-completos

- Dada la siguiente instancia I del TSP:
  1. Matriz de distancias $A \in \mathbb{R}^{n \times n}$.
  2. Número real $k \in \mathbb{R}$.
- ... construimos la siguiente instancia I' del VRP:
  1. Matriz de distancias $A \in \mathbb{R}^{n \times n}$.
  2. Cantidad $m = 1$ de camiones.
  3. Cada cliente $i = 1, \ldots, n$ tiene capacidad $c_i = 1$.
  4. El único camión tiene capacidad $C = n$.
  5. El número real k es el mismo.

Problemas NP-completos

- Dada la siguiente instancia I del TSP:
  1. Matriz de distancias $A \in \mathbb{R}^{n \times n}$.
  2. Número real $k \in \mathbb{R}$.
- ... construimos la siguiente instancia I' del VRP:
  1. Matriz de distancias $A \in \mathbb{R}^{n \times n}$.
  2. Cantidad $m = 1$ de camiones.
  3. Cada cliente $i = 1, \ldots, n$ tiene capacidad $c_i = 1$.
  4. El único camión tiene capacidad $C = n$.
  5. El número real k es el mismo.
- Entonces, la instancia I del TSP tiene distancia total $\leq k$ si y sólo si la instancia I' del VRP tiene distancia total $\leq k$. Esto prueba que VRP es NP-completo.

Problemas NP-completos

- Para demostrar que un problema B es NP-completo, conviene buscar algún problema NP-completo C conocido, y tratar de encontrar una transformación polinomial de C a B.

Problemas NP-completos

- Para demostrar que un problema B es NP-completo, conviene buscar algún problema NP-completo C conocido, y tratar de encontrar una transformación polinomial de C a B.
- Ahora bien, ¿cómo se hizo la primera demostración de NP-completitutd?

Problemas NP-completos

- Para demostrar que un problema B es NP-completo, conviene buscar algún problema NP-completo C conocido, y tratar de encontrar una transformación polinomial de C a B.
- Ahora bien, ¿cómo se hizo la primera demostración de NP-completitutd?

  Satisfactibilidad (SAT)

Problemas NP-completos

- Para demostrar que un problema B es NP-completo, conviene buscar algún problema NP-completo C conocido, y tratar de encontrar una transformación polinomial de C a B.
- Ahora bien, ¿cómo se hizo la primera demostración de NP-completitutd?

SATISFACTIBILIDAD (SAT)

1. **Entrada:** Una fórmula proposicional f.

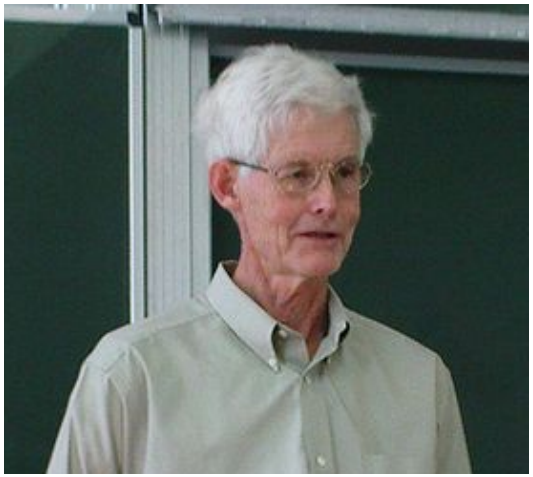
Problemas NP-completos

- Para demostrar que un problema B es NP-completo, conviene buscar algún problema NP-completo C conocido, y tratar de encontrar una transformación polinomial de C a B.
- Ahora bien, ¿cómo se hizo la primera demostración de NP-completitutd?

SATISFACTIBILIDAD (SAT)

1. **Entrada:** Una fórmula proposicional f.
2. **Salida:** ¿Existe una asignación de valores de verdad a las proposiciones de f que hace que f sea verdadera?

Problemas NP-completos



Stephen Cook



Leonid Levin

- **Teorema (Cook, 1971 – Levin, 1973).**
  SAT es NP-completo.

Problemas NP-completos



Richard Karp

- A partir del Teorema de Cook-Levin, Richard Karp demostró en 1972 que otros 21 problemas son NP-completos.

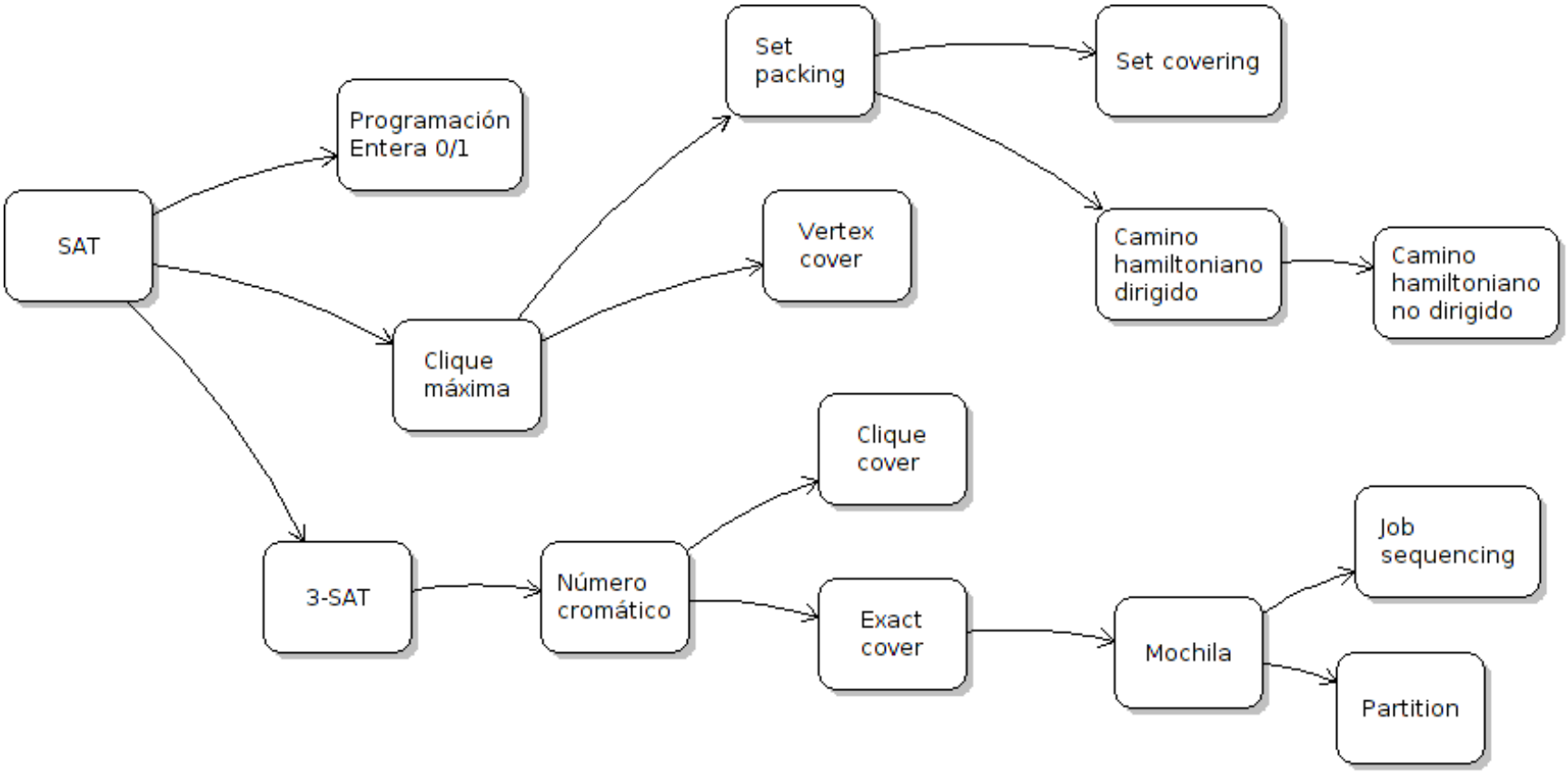
Problemas NP-completos



Richard Karp

- A partir del Teorema de Cook-Levin, Richard Karp demostró en 1972 que otros 21 problemas son NP-completos.
- Actualmente se conocen más de 3.000 problemas NP-completos!

Problemas NP-completos



Problemas NP-completos

- **Ejemplo:** Reducción de 3-SAT a conjunto independiente máximo.

Problemas NP-completos

- **Ejemplo:** Reducción de 3-SAT a conjunto independiente máximo.

  3-SAT

Problemas NP-completos

- **Ejemplo:** Reducción de 3-SAT a conjunto independiente máximo.

  3-SAT

  1. **Entrada:** Una fórmula proposicional con a lo sumo tres proposiciones por cláusula.

Problemas NP-completos

- **Ejemplo:** Reducción de 3-SAT a conjunto independiente máximo.

  3-SAT

  1. **Entrada:** Una fórmula proposicional con a lo sumo tres proposiciones por cláusula.
  2. **Salida:** ¿Existe una valuación que haga verdadera a la fórmula?

Problemas NP-completos

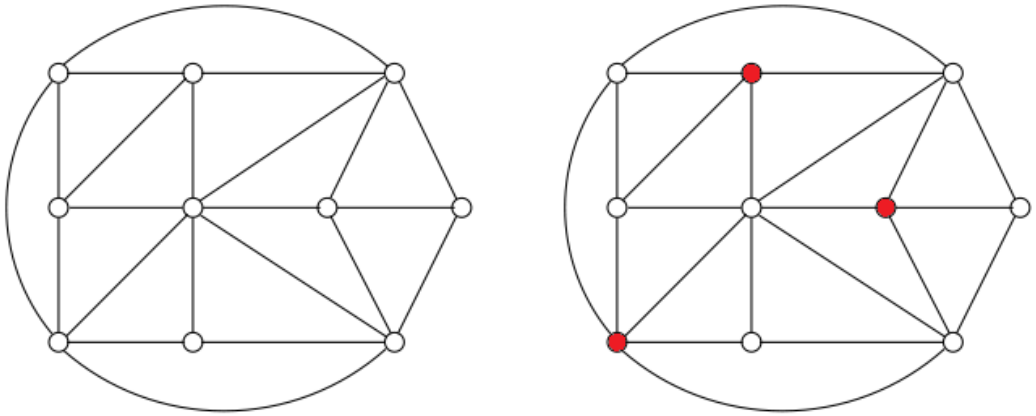
- **Ejemplo:** Reducción de 3-SAT a conjunto independiente máximo.

  3-SAT
  1. **Entrada:** Una fórmula proposicional con a lo sumo tres proposiciones por cláusula.
  2. **Salida:** ¿Existe una valuación que haga verdadera a la fórmula?

- Por ejemplo, la siguiente fórmula tiene no más de tres proposiciones por cláusula:

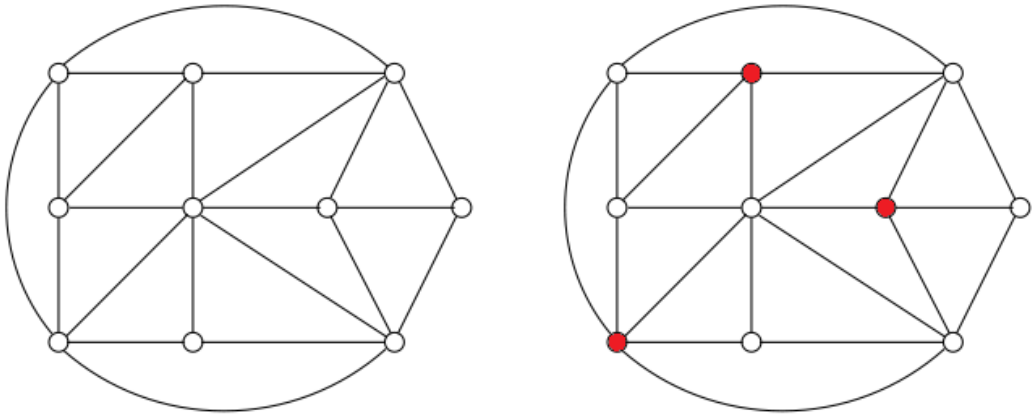
$$(\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y})$$

Problemas NP-completos



- Un conjunto independiente en un grafo es un conjunto de vértices que no son vecinos entre sí.

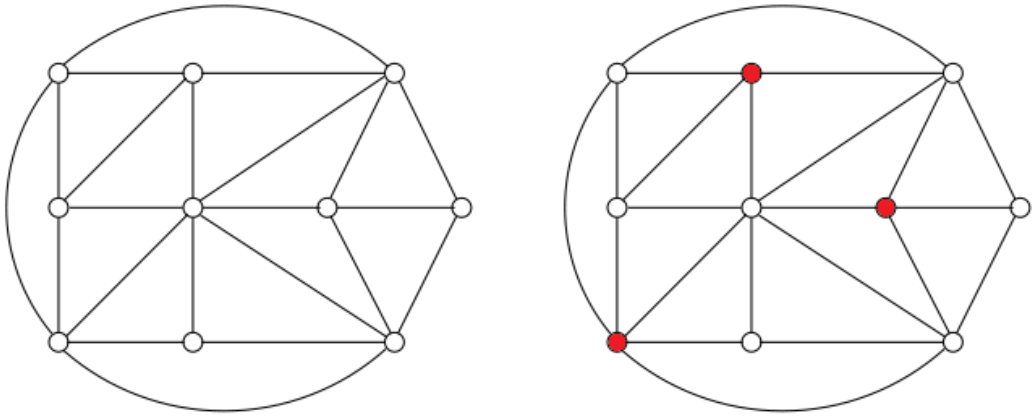
Problemas NP-completos



- Un conjunto independiente en un grafo es un conjunto de vértices que no son vecinos entre sí.

  Conjunto independiente máximo (MIS)

Problemas NP-completos

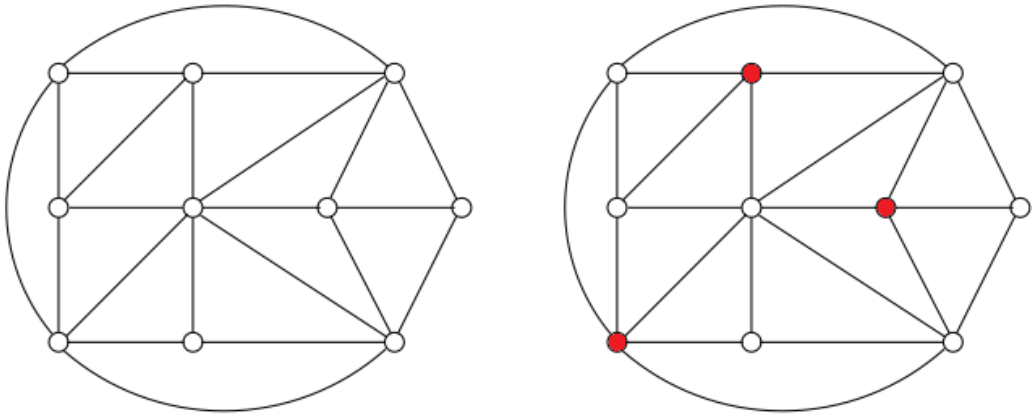


- Un conjunto independiente en un grafo es un conjunto de vértices que no son vecinos entre sí.

Conjunto independiente máximo (MIS)

1. **Entrada:** Un grafo G y un número $k \in \mathbb{Z}_+$.

Problemas NP-completos



- Un conjunto independiente en un grafo es un conjunto de vértices que no son vecinos entre sí.

Conjunto independiente máximo (MIS)

1. **Entrada:** Un grafo G y un número $k \in \mathbb{Z}_+$.
2. **Salida:** ¿Existe un conjunto independiente en G de tamaño k o mayor?

Problemas NP-completos

- Veamos que 3-SAT $\leq_P$ MIS. Dada una instancia I de 3-SAT (es decir, dada una fórmula) construimos un grafo del siguiente modo.

Problemas NP-completos

- Veamos que 3-SAT $\leq_P$ MIS. Dada una instancia I de 3-SAT (es decir, dada una fórmula) construimos un grafo del siguiente modo.
- Para la fórmula ...

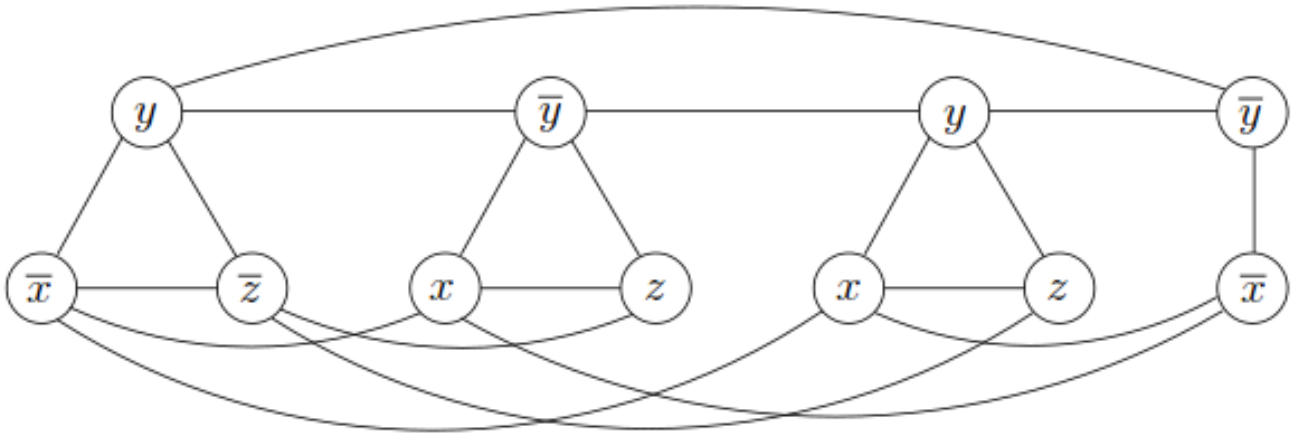
$$(\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y})$$

Problemas NP-completos

- Veamos que 3-SAT $\leq_P$ MIS. Dada una instancia I de 3-SAT (es decir, dada una fórmula) construimos un grafo del siguiente modo.
- Para la fórmula ...

$$(\bar{x} \vee y \vee \bar{z}) \wedge (x \vee \bar{y} \vee z) \wedge (x \vee y \vee z) \wedge (\bar{x} \vee \bar{y})$$

- ... construimos el grafo ...



Problemas NP-completos

- Si la fórmula tiene c cláusulas ($c = 4$ en el ejemplo), entonces fijamos $k = c$.

Problemas NP-completos

- Si la fórmula tiene c cláusulas ($c = 4$ en el ejemplo), entonces fijamos $k = c$.
- **Teorema.** La fórmula es satisfactible si y sólo si el grafo tiene un conjunto independiente de tamaño k o superior.

Problemas NP-completos

- Si la fórmula tiene c cláusulas ($c = 4$ en el ejemplo), entonces fijamos $k = c$.
- **Teorema.** La fórmula es satisfactible si y sólo si el grafo tiene un conjunto independiente de tamaño k o superior.
- Esto muestra que 3-SAT $\leq_P$ MIS. Como 3-SAT es NP-completo, entonces MIS también es NP-completo.

Problemas NP-completos

- Dado un problema de decisión, tenemos tres posibilidades:

Problemas NP-completos

- Dado un problema de decisión, tenemos tres posibilidades:

  1. Existe un algoritmo polinomial para el problema (y se demuestra encontrando un algoritmo polinomial que lo resuelve).

Problemas NP-completos

- Dado un problema de decisión, tenemos tres posibilidades:
  1. Existe un algoritmo polinomial para el problema (y se demuestra encontrando un algoritmo polinomial que lo resuelve).
  2. El problema es NP-completo (y se demuestra a través de una transformación polinomial desde otro problema NP-completo).

Problemas NP-completos

- Dado un problema de decisión, tenemos tres posibilidades:
  1. Existe un algoritmo polinomial para el problema (y se demuestra encontrando un algoritmo polinomial que lo resuelve).
  2. El problema es NP-completo (y se demuestra a través de una transformación polinomial desde otro problema NP-completo).
  3. Es un problema abierto!

Problemas NP-completos

- ¿Qué hacemos si tenemos que resolver un problema NP-completo?

Problemas NP-completos

- ¿Qué hacemos si tenemos que resolver un problema NP-completo?

  1. Analizar si el tamaño de las instancias a resolver permite un enfoque basado en fuerza bruta o backtracking.

Problemas NP-completos

- ¿Qué hacemos si tenemos que resolver un problema NP-completo?
  1. Analizar si el tamaño de las instancias a resolver permite un enfoque basado en fuerza bruta o backtracking.
  2. Analizar si se puede reducir a un problema NP-completo que tenga solvers eficientes en la práctica (como SAT o programación lineal entera).

Problemas NP-completos

- ¿Qué hacemos si tenemos que resolver un problema NP-completo?
  1. Analizar si el tamaño de las instancias a resolver permite un enfoque basado en fuerza bruta o backtracking.
  2. Analizar si se puede reducir a un problema NP-completo que tenga solvers eficientes en la práctica (como SAT o programación lineal entera).
  3. Buscar casos particulares que se puedan resolver en forma eficiente. Es habitual que los problemas NP-completos tengan casos particulares polinomiales (habitualmente dados por restricciones sobre el input).

Problemas NP-completos

- ¿Qué hacemos si tenemos que resolver un problema NP-completo?
  1. Analizar si el tamaño de las instancias a resolver permite un enfoque basado en fuerza bruta o backtracking.
  2. Analizar si se puede reducir a un problema NP-completo que tenga solvers eficientes en la práctica (como SAT o programación lineal entera).
  3. Buscar casos particulares que se puedan resolver en forma eficiente. Es habitual que los problemas NP-completos tengan casos particulares polinomiales (habitualmente dados por restricciones sobre el input).
  4. Diseñar heurísticas ad hoc para el problema, tratando de aprovechar su estructura particular.

Problemas NP-completos

- ¿Qué hacemos si tenemos que resolver un problema NP-completo?
  1. Analizar si el tamaño de las instancias a resolver permite un enfoque basado en fuerza bruta o backtracking.
  2. Analizar si se puede reducir a un problema NP-completo que tenga solvers eficientes en la práctica (como SAT o programación lineal entera).
  3. Buscar casos particulares que se puedan resolver en forma eficiente. Es habitual que los problemas NP-completos tengan casos particulares polinomiales (habitualmente dados por restricciones sobre el input).
  4. Diseñar heurísticas ad hoc para el problema, tratando de aprovechar su estructura particular.
  5. Diseñar metaheurísticas para el problema, como algoritmos genéticos o algoritmos evolutivos.

La conjetura P vs. NP

- Si existe un algoritmo polinomial para algún problema NP-completo, entonces P = NP y todos los problemas en NP son polinomiales.

La conjetura P vs. NP

- Si existe un algoritmo polinomial para algún problema NP-completo, entonces P = NP y todos los problemas en NP son polinomiales.
- Por el contrario, si se prueba que no existe ningún algoritmo polinomial para algún problema NP-completo, entonces P $\neq$ NP y todos los problemas NP-completos son exponenciales.

La conjetura P vs. NP

- Si existe un algoritmo polinomial para algún problema NP-completo, entonces P = NP y todos los problemas en NP son polinomiales.
- Por el contrario, si se prueba que no existe ningún algoritmo polinomial para algún problema NP-completo, entonces P $\neq$ NP y todos los problemas NP-completos son exponenciales.
- No se sabe cuál es la situación, y este problema abierto ha recibido mucha atención en las últimas décadas.

La conjetura P vs. NP

A proof that $P = NP$ could have stunning practical consequences, if the proof leads to efficient methods for solving some of the important problems in NP. It is also possible that a proof would not lead directly to efficient methods, perhaps if the proof is non-constructive, or the size of the bounding polynomial is too big to be efficient in practice. The consequences, both positive and negative, arise since various NP-complete problems are fundamental in many fields.

Cryptography, for example, relies on certain problems being difficult. A constructive and efficient solution to an NP-complete problem such as 3-SAT would break most existing cryptosystems including public-key cryptography, a foundation for many modern security applications such as secure economic transactions over the Internet, and symmetric ciphers such as AES or 3DES, used for the encryption of communications data. These would need to be modified or replaced by information-theoretically secure solutions.

La conjetura P vs. NP

On the other hand, there are enormous positive consequences that would follow from rendering tractable many currently mathematically intractable problems. For instance, many problems in operations research are NP-complete, such as some types of integer programming, and the travelling salesman problem, to name two of the most famous examples. Efficient solutions to these problems would have enormous implications for logistics. Many other important problems, such as some problems in protein structure prediction, are also NP-complete; if these problems were efficiently solvable it could spur considerable advances in biology.

A proof that showed that $P \neq NP$ would lack the practical computational benefits of a proof that $P = NP$, but would nevertheless represent a very significant advance in computational complexity theory and provide guidance for future research. It would allow one to show in a formal way that many common problems cannot be solved efficiently, so that the attention of researchers can be focused on partial solutions or solutions to other problems. Due to widespread belief in $P \neq NP$, much of this focusing of research has already taken place.